Internationalization in Java

Learning by Example

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-15

Outline

1. Locales
2. Numbers and Money
3. Dates and Times
4. Translated Messages
5. Working with Text
6. Reading Input
7. Summary

How to read these slides

- Every program is a file in `codeoutput/`; the output is what it really printed with Java 21.
- The programs run on an English (US) system in Istanbul, unless the command line says otherwise (`-Duser.language=tr`, `LANG=tr_TR.UTF-8`).
- The code comes first; the output appears on the next click. Predict it first.

Windows (Command Prompt)

```
chcp 65001
javac -encoding UTF-8 -d build codeoutput\Numbers.
      java
java -Dstdout.encoding=UTF-8 -cp build Numbers
```

macOS (Terminal)

```
javac -d build codeoutput/Numbers.java
java -cp build Numbers
bash run_examples.sh          # all of them
```

`chcp 65001` switches the Windows console to UTF-8, so Turkish letters print correctly.

Locales

Making locales

codeoutput/Locales.java — Java

```java
import java.util.Locale;

public class Locales {
    public static void main(String[] args) {
        Locale[] all = {
            Locale.of("tr", "TR"),              // Java 19+
            Locale.forLanguageTag("de-CH"),     // from a BCP 47 tag
            Locale.FRANCE,                      // a constant
            Locale.forLanguageTag("pt-BR"),
            new Locale.Builder().setLanguage("sr").setScript("Latn")
                                .setRegion("RS").build()
        };
        for (Locale l : all)
            System.out.printf("%-8s %-11s %-30s %s%n", l, l.toLanguageTag(),
                l.getDisplayName(Locale.ENGLISH), l.getDisplayName(l));
    }
}
```

`toLanguageTag()` gives the standard tag; `getDisplayName(l)` is what a language menu should show.

Making locales

codeoutput/Locales.java — Java

```java
import java.util.Locale;

public class Locales {
    public static void main(String[] args) {
        Locale[] all = {
            Locale.of("tr", "TR"),              // Java 19+
            Locale.forLanguageTag("de-CH"),     // from a BCP 47 tag
            Locale.FRANCE,                      // a constant
            Locale.forLanguageTag("pt-BR"),
            new Locale.Builder().setLanguage("sr").setScript("Latn")
                                .setRegion("RS").build()
        };
        for (Locale l : all)
            System.out.printf("%-8s %-11s %-30s %s%n", l, l.toLanguageTag(),
                l.getDisplayName(Locale.ENGLISH), l.getDisplayName(l));
    }
}
```

Terminal — codeoutput/Locales.txt

```
tr_TR    tr-TR       Turkish (Türkiye)             Türkçe (Türkiye)
de_CH    de-CH       German (Switzerland)          Deutsch (Schweiz)
fr_FR    fr-FR       French (France)               français (France)
pt_BR    pt-BR       Portuguese (Brazil)           português (Brasil)
sr_RS_#Latn sr-Latn-RS  Serbian (Latin, Serbia)       srpski (latinica, Srbija)
```

`toLanguageTag()` gives the standard tag; `getDisplayName(l)` is what a language menu should show.

The default locale

codeoutput/Default/Default.java **Java**

```java
import java.util.Locale;

public class Default {
    public static void main(String[] args) {
        System.out.println("default locale: " + Locale.
        getDefault());
        System.out.printf("pi = %.2f, big = %,d%n", Math
        .PI, 1234567);
        System.out.println(String.format(Locale.ROOT, "
        fixed: %.2f", Math.PI));
    }
}
```

The same program, three answers. `LANG` is how macOS and Linux pass the regional setting; on **Windows** Java reads the *Regional format* of the user. `-D` overrides both. Locale.ROOT gives the same output everywhere.

The default locale

codeoutput/Default/Default.java — Java

```java
import java.util.Locale;

public class Default {
    public static void main(String[] args) {
        System.out.println("default locale: " + Locale.getDefault());
        System.out.printf("pi = %.2f, big = %,d%n", Math.PI, 1234567);
        System.out.println(String.format(Locale.ROOT, "fixed: %.2f", Math.PI));
    }
}
```

Terminal — codeoutput/Default.txt

```
$ java Default.java
default locale: en_US
pi = 3.14, big = 1,234,567
fixed: 3.14
$ LANG=tr_TR.UTF-8 java Default.java
default locale: tr_TR
pi = 3,14, big = 1.234.567
fixed: 3.14
$ java -Duser.language=de -Duser.country=DE Default.java
default locale: de_DE
pi = 3,14, big = 1.234.567
fixed: 3.14
```

The same program, three answers. `LANG` is how macOS and Linux pass the regional setting; on **Windows** Java reads the *Regional format* of the user. `-D` overrides both. Locale.ROOT gives the same output everywhere.

Numbers and Money

Numbers in five locales

codeoutput/Numbers.java — Java

```java
import java.text.NumberFormat;
import java.util.Locale;

public class Numbers {
    public static void main(String[] args) {
        double x = 1234567.891;
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE", "fr-FR", "de-CH"}) {
            Locale l = Locale.forLanguageTag(tag);
            System.out.printf("%-6s %-14s %-11s %-5s %s%n", tag,
                NumberFormat.getNumberInstance(l).format(x),
                NumberFormat.getIntegerInstance(l).format(x),
                NumberFormat.getPercentInstance(l).format(0.25),
                NumberFormat.getCurrencyInstance(l).format(x));
        }
    }
}
```

One value, five spellings: French groups with a narrow space, Swiss German with ’; Turkish writes %25.

Numbers in five locales

codeoutput/Numbers.java — Java

```java
import java.text.NumberFormat;
import java.util.Locale;

public class Numbers {
    public static void main(String[] args) {
        double x = 1234567.891;
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE", "fr-FR", "de-CH"}) {
            Locale l = Locale.forLanguageTag(tag);
            System.out.printf("%-6s %-14s %-11s %-5s %s%n", tag,
                NumberFormat.getNumberInstance(l).format(x),
                NumberFormat.getIntegerInstance(l).format(x),
                NumberFormat.getPercentInstance(l).format(0.25),
                NumberFormat.getCurrencyInstance(l).format(x));
        }
    }
}
```

Terminal — codeoutput/Numbers.txt

```
en-US  1,234,567.891  1,234,568   25%   $1,234,567.89
tr-TR  1.234.567,891  1.234.568   %25   ₺1.234.567,89
de-DE  1.234.567,891  1.234.568   25 %  1.234.567,89 €
fr-FR  1 234 567,891  1 234 568   25 %  1 234 567,89 €
de-CH  1’234’567.891  1’234’568   25%   CHF 1’234’567.89
```

One value, five spellings: French groups with a narrow space, Swiss German with ’; Turkish writes %25.

Currencies

codeoutput/Money.java — Java

```java
import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Currency;
import java.util.Locale;

public class Money {
    public static void main(String[] args) {
        Locale tr = Locale.of("tr", "TR");
        BigDecimal price = new BigDecimal("1234.5");
        for (String code : new String[] {"TRY", "EUR", "USD", "JPY"}) {
            Currency c = Currency.getInstance(code);
            NumberFormat f = NumberFormat.getCurrencyInstance(tr);
            f.setCurrency(c);                          // Turkish format, other currency
            System.out.printf("%s  digits=%d  symbol(en)=%-4s  %-22s %s%n", code,
                c.getDefaultFractionDigits(), c.getSymbol(Locale.US),
                c.getDisplayName(tr), f.format(price));
        }
    }
}
```

Turkish format, other currencies. The yen has no decimals, but `setCurrency` kept two: set them too.

Currencies

codeoutput/Money.java — Java

```java
import java.math.BigDecimal;
import java.text.NumberFormat;
import java.util.Currency;
import java.util.Locale;

public class Money {
    public static void main(String[] args) {
        Locale tr = Locale.of("tr", "TR");
        BigDecimal price = new BigDecimal("1234.5");
        for (String code : new String[] {"TRY", "EUR", "USD", "JPY"}) {
            Currency c = Currency.getInstance(code);
            NumberFormat f = NumberFormat.getCurrencyInstance(tr);
            f.setCurrency(c);                          // Turkish format, other currency
            System.out.printf("%s  digits=%d  symbol(en)=%-4s  %-22s %s%n", code,
                c.getDefaultFractionDigits(), c.getSymbol(Locale.US),
                c.getDisplayName(tr), f.format(price));
        }
    }
}
```

Terminal — codeoutput/Money.txt

```
TRY  digits=2  symbol(en)=TRY   Türk Lirası            ₺1.234,50
EUR  digits=2  symbol(en)=€     Euro                   €1.234,50
USD  digits=2  symbol(en)=$     ABD Doları             $1.234,50
JPY  digits=0  symbol(en)=¥     Japon Yeni             ¥1.234,50
```

Turkish format, other currencies. The yen has no decimals, but `setCurrency` kept two: set them too.

Reading numbers typed by people

codeoutput/Parse.java — Java

```java
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.Locale;

public class Parse {
    public static void main(String[] args) throws ParseException {
        String typed = "1.234,5";                          // as a Turkish user types it
        NumberFormat tr = NumberFormat.getInstance(Locale.of("tr", "TR"));
        NumberFormat us = NumberFormat.getInstance(Locale.US);
        System.out.println("Turkish rules: " + tr.parse(typed));
        System.out.println("US rules:      " + us.parse(typed)); // stops at ','
        System.out.println("parseDouble:   " + Double.parseDouble("3,5"));
    }
}
```

`parse` stops at the first character it does not understand: with US rules `"1.234,5"` is `1.234`.

Reading numbers typed by people

codeoutput/Parse.java — Java

```java
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.Locale;

public class Parse {
    public static void main(String[] args) throws ParseException {
        String typed = "1.234,5";                          // as a Turkish user types it
        NumberFormat tr = NumberFormat.getInstance(Locale.of("tr", "TR"));
        NumberFormat us = NumberFormat.getInstance(Locale.US);
        System.out.println("Turkish rules: " + tr.parse(typed));
        System.out.println("US rules:      " + us.parse(typed)); // stops at ','
        System.out.println("parseDouble:   " + Double.parseDouble("3,5"));
    }
}
```

Terminal — codeoutput/Parse.txt

```
Turkish rules: 1234.5
US rules:      1.234
Exception in thread "main" java.lang.NumberFormatException: For input string: "3,5"
        at java.base/jdk.internal.math.FloatingDecimal.readJavaFormatString(FloatingDecimal.java:2054)
        at java.base/jdk.internal.math.FloatingDecimal.parseDouble(FloatingDecimal.java:110)
        at java.base/java.lang.Double.parseDouble(Double.java:792)
        at Parse.main(Parse.java:12)
```

`parse` stops at the first character it does not understand: with US rules `"1.234,5"` is `1.234`.

Short numbers: 1.2K, 1 Mn

codeoutput/Compact.java — Java

```java
import java.text.NumberFormat;
import java.util.Locale;

public class Compact {
    public static void main(String[] args) {
        long[] values = {1_200, 1_234_567, 3_000_000_000L};
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE"}) {
            Locale l = Locale.forLanguageTag(tag);
            NumberFormat s = NumberFormat.getCompactNumberInstance(l, NumberFormat.Style.SHORT);
            NumberFormat g = NumberFormat.getCompactNumberInstance(l, NumberFormat.Style.LONG);
            StringBuilder line = new StringBuilder(tag);
            for (long v : values)
                line.append(String.format("  %-8s %-13s", s.format(v), g.format(v)));
            System.out.println(line);
        }
    }
}
```

`CompactNumberFormat` (Java 12): for dashboards and social media counts.

Short numbers: 1.2K, 1 Mn

codeoutput/Compact.java — Java

```java
import java.text.NumberFormat;
import java.util.Locale;

public class Compact {
    public static void main(String[] args) {
        long[] values = {1_200, 1_234_567, 3_000_000_000L};
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE"}) {
            Locale l = Locale.forLanguageTag(tag);
            NumberFormat s = NumberFormat.getCompactNumberInstance(l, NumberFormat.Style.SHORT);
            NumberFormat g = NumberFormat.getCompactNumberInstance(l, NumberFormat.Style.LONG);
            StringBuilder line = new StringBuilder(tag);
            for (long v : values)
                line.append(String.format("  %-8s %-13s", s.format(v), g.format(v)));
            System.out.println(line);
        }
    }
}
```

Terminal — codeoutput/Compact.txt

```
en-US  1K       1 thousand     1M       1 million      3B       3 billion
tr-TR  1 B      1 bin          1 Mn     1 milyon       3 Mr     3 milyar
de-DE  1.200    1 Tausend      1 Mio.   1 Million      3 Mrd.   3 Milliarden
```

`CompactNumberFormat` (Java 12): for dashboards and social media counts.

Dates and Times

Dates in three locales

codeoutput/Dates.java — Java

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

public class Dates {
    public static void main(String[] args) {
        LocalDateTime t = LocalDateTime.of(2026, 10, 15, 14, 30);
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE"}) {
            Locale l = Locale.forLanguageTag(tag);
            System.out.println(tag);
            for (FormatStyle s : new FormatStyle[] {FormatStyle.FULL, FormatStyle.MEDIUM,
                                                    FormatStyle.SHORT})
                System.out.printf("  %-6s %s%n", s, t.format(
                    DateTimeFormatter.ofLocalizedDate(s).withLocale(l)));
            System.out.println("  time    " + t.format(
                DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT).withLocale(l)));
        }
        System.out.println(t.format(DateTimeFormatter.ofPattern("EEEE, d MMMM yyyy",
                                                                 Locale.of("tr"))));
    }
}
```

`ofLocalizedDate`: order, names, separators of the locale. `ofPattern` still needs a locale for names.

Dates in three locales

codeoutput/Dates.java — Java

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

public class Dates {
    public static void main(String[] args) {
        LocalDateTime t = LocalDateTime.of(2026, 10, 15, 14, 30);
        for (String tag : new String[] {"en-US", "tr-TR", "de-DE"}) {
            Locale l = Locale.forLanguageTag(tag);
            System.out.println(tag);
            for (FormatStyle s : new FormatStyle[] {FormatStyle.FULL, FormatStyle.MEDIUM,
                                                    FormatStyle.SHORT})
                System.out.printf("  %-6s %s%n", s, t.format(
                    DateTimeFormatter.ofLocalizedDate(s).withLocale(l)));
            System.out.println("  time   " + t.format(
                DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT).withLocale(l)));
        }
        System.out.println(t.format(DateTimeFormatter.ofPattern("EEEE, d MMMM yyyy",
                                                                 Locale.of("tr"))));
    }
}
```

Terminal codeoutput/Dates.txt

```
en-US
  FULL   Thursday, October 15, 2026
  MEDIUM Oct 15, 2026
  SHORT  10/15/26
  time   2:30 PM
tr-TR
  FULL   15 Ekim 2026 Perşembe
  MEDIUM 15 Eki 2026
  SHORT  15.10.2026
  time   14:30
de-DE
  FULL   Donnerstag, 15. Oktober 2026
  MEDIUM 15.10.2026
  SHORT  15.10.26
  time   14:30
Perşembe, 15 Ekim 2026
```

`ofLocalizedDate`: order, names, separators of the locale. `ofPattern` still needs a locale for names.

One moment, four time zones

codeoutput/Zones.java — Java

```java
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public class Zones {
    public static void main(String[] args) {
        ZonedDateTime meeting = ZonedDateTime.of(2026, 10, 15, 14, 30, 0, 0,
                                                 ZoneId.of("Europe/Istanbul"));
        for (String z : new String[] {"Europe/Istanbul", "Europe/London",
                                      "America/New_York", "Asia/Tokyo"})
            System.out.println(meeting.withZoneSameInstant(ZoneId.of(z)));

        // 02:30 does not exist in New York on 8 March 2026: clocks jump 02:00 -> 03:00
        System.out.println(ZonedDateTime.of(LocalDateTime.of(2026, 3, 8, 2, 30),
                                            ZoneId.of("America/New_York")));
    }
}
```

Same moment, other wall clocks. Last line: 02:30 does not exist that night (daylight saving), so 03:30.

One moment, four time zones

codeoutput/Zones.java — Java

```java
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZonedDateTime;

public class Zones {
    public static void main(String[] args) {
        ZonedDateTime meeting = ZonedDateTime.of(2026, 10, 15, 14, 30, 0, 0,
                                                 ZoneId.of("Europe/Istanbul"));
        for (String z : new String[] {"Europe/Istanbul", "Europe/London",
                                      "America/New_York", "Asia/Tokyo"})
            System.out.println(meeting.withZoneSameInstant(ZoneId.of(z)));

        // 02:30 does not exist in New York on 8 March 2026: clocks jump 02:00 -> 03:00
        System.out.println(ZonedDateTime.of(LocalDateTime.of(2026, 3, 8, 2, 30),
                                            ZoneId.of("America/New_York")));
    }
}
```

Terminal — codeoutput/Zones.txt

```
2026-10-15T14:30+03:00[Europe/Istanbul]
2026-10-15T12:30+01:00[Europe/London]
2026-10-15T07:30-04:00[America/New_York]
2026-10-15T20:30+09:00[Asia/Tokyo]
2026-03-08T03:30-04:00[America/New_York]
```

Same moment, other wall clocks. Last line: 02:30 does not exist that night (daylight saving), so 03:30.

Translated Messages

A resource bundle

codeoutput/Bundle/Hello.java **Java**

```java
import java.text.MessageFormat;
import java.util.Locale;
import java.util.ResourceBundle;

public class Hello {
    public static void main(String[] args) {
        Locale l = Locale.forLanguageTag(args[0]);
        ResourceBundle rb = ResourceBundle.getBundle("Messages", l);
        System.out.println("[" + rb.getLocale() + "] "
            + MessageFormat.format(rb.getString("greet"), "Ayşe") + " | "
            + rb.getString("save") + " | " + rb.getString("quit"));
    }
}
```

codeoutput/Bundle/Messages.properties **properties**

```properties
greet=Hello, {0}!
save=Save
quit=Quit
```

codeoutput/Bundle/Messages_tr.properties **properties**

```properties
greet=Merhaba, {0}!
save=Kaydet
quit=Çıkış
```

codeoutput/Bundle/Messages_de.properties **properties**

```properties
greet=Hallo, {0}!
save=Speichern
```

Which file answers?

Terminal codeoutput/Bundle.txt

```
$ ls
Hello.java
Messages_de.properties
Messages.properties
Messages_tr.properties
$ java Hello.java tr
[tr] Merhaba, Ayşe! | Kaydet | Çıkış
$ java Hello.java de
[de] Hallo, Ayşe! | Speichern | Quit
$ java Hello.java fr
[] Hello, Ayşe! | Save | Quit
$ java Hello.java en-GB
[] Hello, Ayşe! | Save | Quit
```

de: `quit` is missing in German, so it comes from the base file. `fr` and `en-GB`: no file, so the **default locale** is tried (`en_US`, no file either) and then the base file: `rb.getLocale()` is empty.

Values in messages: MessageFormat

codeoutput/Messages.java — Java

```java
import java.text.MessageFormat;
import java.time.LocalDate;
import java.util.Locale;

public class Messages {
    public static void main(String[] args) {
        Object[] v = {"Ali", 3, 1234.5};
        String en = "{0} sent {1} files ({2,number,#,##0.0} KB).";
        String tr = "{0}, {1} dosya gönderdi ({2,number,#,##0.0} KB).";
        System.out.println(new MessageFormat(en, Locale.US).format(v));
        System.out.println(new MessageFormat(tr, Locale.of("tr")).format(v));

        String plural = "{0,choice,0#no files|1#one file|1<{0,number,integer} files}";
        for (int n : new int[] {0, 1, 2, 1500})
            System.out.println(new MessageFormat(plural, Locale.US).format(new Object[] {n}))
        ;

        System.out.println(MessageFormat.format("Don't open {0}", "x.txt"));  // ' is special
        System.out.println(MessageFormat.format("Don''t open {0}", "x.txt"));
    }
}
```

Each language puts {0}, {1} where its grammar wants them. `choice` selects a text by number. A single ’ quotes, so write '' for an apostrophe.

Values in messages: MessageFormat

codeoutput/Messages.java — Java

```java
import java.text.MessageFormat;
import java.time.LocalDate;
import java.util.Locale;

public class Messages {
    public static void main(String[] args) {
        Object[] v = {"Ali", 3, 1234.5};
        String en = "{0} sent {1} files ({2,number,#,##0.0} KB).";
        String tr = "{0}, {1} dosya gönderdi ({2,number,#,##0.0} KB).";
        System.out.println(new MessageFormat(en, Locale.US).format(v));
        System.out.println(new MessageFormat(tr, Locale.of("tr")).format(v));

        String plural = "{0,choice,0#no files|1#one file|1<{0,number,integer} files}";
        for (int n : new int[] {0, 1, 2, 1500})
            System.out.println(new MessageFormat(plural, Locale.US).format(new Object[] {n}))
        ;

        System.out.println(MessageFormat.format("Don't open {0}", "x.txt"));  // ' is special
        System.out.println(MessageFormat.format("Don''t open {0}", "x.txt"));
    }
}
```

Terminal
codeoutput/Messages.txt

```
Ali sent 3 files (1,234.5 KB).
Ali, 3 dosya gönderdi (1.234,5 KB).
no files
one file
2 files
1,500 files
Dont open {0}
Don't open x.txt
```

Each language puts {0}, {1} where its grammar wants them. `choice` selects a text by number. A single ’ quotes, so write '' for an apostrophe.

Working with Text

Upper and lower case

codeoutput/Case.java — Java

```java
import java.util.Locale;

public class Case {
    public static void main(String[] args) {
        Locale tr = Locale.of("tr", "TR");
        System.out.println("title  -> " + "title".toUpperCase(tr) + "   " + "title".toUpperCase(Locale.ROOT));
        System.out.println("TITLE  -> " + "TITLE".toLowerCase(tr) + "   " + "TITLE".toLowerCase(Locale.ROOT));
        System.out.println("FILE equals file? " + "FILE".toLowerCase(tr).equals("file"));
        System.out.println("straße -> " + "straße".toUpperCase(Locale.GERMANY));
        System.out.println("ılık   -> " + "ılık".toUpperCase(tr) + "   İSTANBUL -> " + "İSTANBUL".toLowerCase(tr));
    }
}
```

Turkish has `i`/`İ` and `ı`/`I`. `"FILE".toLowerCase()` on a Turkish computer is `"fıle"`: never use it to compare keywords; use `Locale.ROOT`.

Upper and lower case

codeoutput/Case.java — Java

```java
import java.util.Locale;

public class Case {
    public static void main(String[] args) {
        Locale tr = Locale.of("tr", "TR");
        System.out.println("title  -> " + "title".toUpperCase(tr) + "   " + "title".toUpperCase(Locale.ROOT));
        System.out.println("TITLE  -> " + "TITLE".toLowerCase(tr) + "   " + "TITLE".toLowerCase(Locale.ROOT));
        System.out.println("FILE equals file? " + "FILE".toLowerCase(tr).equals("file"));
        System.out.println("straße -> " + "straße".toUpperCase(Locale.GERMANY));
        System.out.println("ılık   -> " + "ılık".toUpperCase(tr) + "   İSTANBUL -> " + "İSTANBUL".toLowerCase(tr));
    }
}
```

Terminal — codeoutput/Case.txt

```
title  -> TİTLE   TITLE
TITLE  -> tıtle   title
FILE equals file? false
straße -> STRASSE
ılık   -> ILIK   İSTANBUL -> istanbul
```

Turkish has `i`/`İ` and `ı`/`I`. `"FILE".toLowerCase()` on a Turkish computer is `"fıle"`: never use it to compare keywords; use `Locale.ROOT`.

Sorting words

codeoutput/Sort.java — Java

```java
import java.text.Collator;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class Sort {
    public static void main(String[] args) {
        List<String> words = new ArrayList<>(List.of(
            "çay", "cam", "zil", "şeker", "sabun", "ılık", "ilaç", "Ömer", "ok", "Ali"));
        words.sort(null);                                   // compareTo: code points
        System.out.println(words);
        words.sort(Collator.getInstance(Locale.of("tr")));  // Turkish alphabet
        System.out.println(words);
        words.sort(Collator.getInstance(Locale.ENGLISH));   // English rules
        System.out.println(words);
    }
}
```

1: code points (Ö, Ç, I, Ş last). 2: Turkish alphabet. 3: English rules (I last).

Sorting words

codeoutput/Sort.java — Java

```java
import java.text.Collator;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class Sort {
    public static void main(String[] args) {
        List<String> words = new ArrayList<>(List.of(
            "çay", "cam", "zil", "şeker", "sabun", "ılık", "ilaç", "Ömer", "ok", "Ali"));
        words.sort(null);                                   // compareTo: code points
        System.out.println(words);
        words.sort(Collator.getInstance(Locale.of("tr")));  // Turkish alphabet
        System.out.println(words);
        words.sort(Collator.getInstance(Locale.ENGLISH));   // English rules
        System.out.println(words);
    }
}
```

Terminal — codeoutput/Sort.txt

```
[Ali, cam, ilaç, ok, sabun, zil, Ömer, çay, ılık, şeker]
[Ali, cam, çay, ılık, ilaç, ok, Ömer, sabun, şeker, zil]
[Ali, cam, çay, ilaç, ok, Ömer, sabun, şeker, zil, ılık]
```

1: code points (Ö, ç, ı, ş last). 2: Turkish alphabet. 3: English rules (ı last).

Two ways to write é

codeoutput/Normal.java — Java

```java
import java.text.Normalizer;

public class Normal {
    static String hex(String s) {                    // the code points, e.g. "43 61 66 E9"
        StringBuilder b = new StringBuilder();
        s.codePoints().forEach(c -> b.append(String.format("%X ", c)));
        return b.toString();
    }
    public static void main(String[] args) {
        String one = "Caf\u00e9";                    // é as one code point
        String two = "Cafe\u0301";                   // e + combining acute accent
        System.out.println("one: " + hex(one) + "  length " + one.length());
        System.out.println("two: " + hex(two) + "  length " + two.length());
        System.out.println("equal? " + one.equals(two));
        String n1 = Normalizer.normalize(one, Normalizer.Form.NFC);
        String n2 = Normalizer.normalize(two, Normalizer.Form.NFC);
        System.out.println("after NFC: " + hex(n2) + " equal? " + n1.equals(n2));
    }
}
```

They look the same but are different strings until both are normalized to NFC.

Two ways to write é

codeoutput/Normal.java — Java

```java
import java.text.Normalizer;

public class Normal {
    static String hex(String s) {                    // the code points, e.g. "43 61 66 E9"
        StringBuilder b = new StringBuilder();
        s.codePoints().forEach(c -> b.append(String.format("%X ", c)));
        return b.toString();
    }
    public static void main(String[] args) {
        String one = "Caf\u00e9";              // é as one code point
        String two = "Cafe\u0301";             // e + combining acute accent
        System.out.println("one: " + hex(one) + "  length " + one.length());
        System.out.println("two: " + hex(two) + "  length " + two.length());
        System.out.println("equal? " + one.equals(two));
        String n1 = Normalizer.normalize(one, Normalizer.Form.NFC);
        String n2 = Normalizer.normalize(two, Normalizer.Form.NFC);
        System.out.println("after NFC: " + hex(n2) + " equal? " + n1.equals(n2));
    }
}
```

Terminal — codeoutput/Normal.txt

```
one: 43 61 66 E9   length 4
two: 43 61 66 65 301   length 5
equal? false
after NFC: 43 61 66 E9  equal? true
```

They look the same but are different strings until both are normalized to NFC.

Strings and bytes

codeoutput/Bytes.java — Java

```java
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public class Bytes {
    public static void main(String[] args) {
        String s = "ağaç";
        for (String cs : new String[] {"UTF-8", "ISO-8859-9", "UTF-16BE"}) {
            StringBuilder hex = new StringBuilder();
            for (byte b : s.getBytes(Charset.forName(cs)))
                hex.append(String.format("%02X ", b));
            System.out.printf("%-10s %s%n", cs, hex);
        }
        byte[] utf8 = s.getBytes(StandardCharsets.UTF_8);
        System.out.println("read as Windows-1252: " + new String(utf8, Charset.forName("windows-1252")));
        System.out.println("read as UTF-8:        " + new String(utf8, StandardCharsets.UTF_8));
    }
}
```

ğ: 2 bytes in UTF-8, 1 in ISO-8859-9. The wrong charset gives *mojibake*: always name it.

Strings and bytes

codeoutput/Bytes.java — Java

```java
import java.nio.charset.Charset;
import java.nio.charset.StandardCharsets;

public class Bytes {
    public static void main(String[] args) {
        String s = "ağaç";
        for (String cs : new String[] {"UTF-8", "ISO-8859-9", "UTF-16BE"}) {
            StringBuilder hex = new StringBuilder();
            for (byte b : s.getBytes(Charset.forName(cs)))
                hex.append(String.format("%02X ", b));
            System.out.printf("%-10s %s%n", cs, hex);
        }
        byte[] utf8 = s.getBytes(StandardCharsets.UTF_8);
        System.out.println("read as Windows-1252: " + new String(utf8, Charset.forName("windows-1252")));
        System.out.println("read as UTF-8:        " + new String(utf8, StandardCharsets.UTF_8));
    }
}
```

Terminal — codeoutput/Bytes.txt

```
UTF-8      61 C4 9F 61 C3 A7
ISO-8859-9 61 F0 61 E7
UTF-16BE   00 61 01 1F 00 61 00 E7
read as Windows-1252: aÄŸaÃ§
read as UTF-8:        ağaç
```

ğ: 2 bytes in UTF-8, 1 in ISO-8859-9. The wrong charset gives *mojibake*: always name it.

Reading Input

Scanner uses the default locale

codeoutput/Scan/Average.java **Java**

```java
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);   // uses the
        default locale!
        double a = in.nextDouble(), b = in.nextDouble();
        System.out.println("average = " + (a + b) / 2);
    }
}
```

On a Turkish computer `nextDouble` expects `3,5`. A program that reads *data files* should fix the locale: `new Scanner(in).useLocale(Locale.ROOT)`. A program that reads what *people* type may keep the default.

Scanner uses the default locale

codeoutput/Scan/Average.java — Java

```java
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);   // uses the default locale!
        double a = in.nextDouble(), b = in.nextDouble();
        System.out.println("average = " + (a + b) / 2);
    }
}
```

Terminal — codeoutput/Scan.txt

```
$ echo "3.5 4" | java Average.java
average = 3.75
$ echo "3.5 4" | LANG=tr_TR.UTF-8 java Average.java 2>&1 | head -2
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
$ echo "3,5 4" | LANG=tr_TR.UTF-8 java Average.java
average = 3.75
```

On a Turkish computer `nextDouble` expects `3,5`. A program that reads *data files* should fix the locale: `new Scanner(in).useLocale(Locale.ROOT)`. A program that reads what *people* type may keep the default.

Summary

What the examples showed

task	use
choose a locale	`Locale.of("tr", "TR")`, `Locale.forLanguageTag("tr-TR")`
machine-readable output	`String.format(Locale.ROOT, ...)`
numbers, money	`NumberFormat.getNumberInstance/getCurrencyInstance(l)`, `BigDecimal`
read numbers	`NumberFormat.getInstance(l).parse(s)`; `Scanner.useLocale(l)`
dates	`DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).withLocale(l)`
time zones	`ZonedDateTime`, `withZoneSameInstant`
texts	`ResourceBundle.getBundle("Messages", l)`
texts with values	`MessageFormat`, `{0,choice,...}`
case, sorting	`toUpperCase(l)`, `Collator.getInstance(l)`
compare text	`Normalizer.normalize(s, Form.NFC)`
bytes	`getBytes(StandardCharsets.UTF_8)`, `new String(b, cs)`

Exercises

1. Run `Numbers` for `en-IN`, `es-MX` and `sv-SE`. What is different?
2. Change `Money` so that every currency is shown in its home locale.
3. Add `Messages_fr.properties` and run `Hello` with `fr` again.
4. Write the plural message for Turkish. Why does it need only one form?
5. Find three words that `Collator.getInstance(Locale.of("tr"))` sorts differently from `compareTo`.
6. Make `Average` work for both `3.5` and `3,5`, whatever the default locale is.

Questions?